

CONTENTS

Práctico 6 (Formato de instrucción. Manejo de instrucciones) — Arquitectura de Computadoras

FINC UdelaR · Resolución completa y verificada	
Preguntas teóricas	
Ejercicio 1 — CPU RISC de 16 bits, 16 registros	
(a) Formato de instrucción	
(b) Programa en alto nivel	
(c) Ensamblador	
(d) Lenguaje máquina de las primeras 7 instrucciones	
Ejercicio 2 — CPU de 16 bits, 8 registros	
Ejercicio 3 — Compilar BBinaria (búsqueda binaria) en el ensamblador del Ejercicio 2	
Ejercicio 4 — Selection sort en microprocesador de 32 bits, 16 registros	
(a) Alto nivel	
(b) Ensamblador (verificado por simulación)	
Ejercicio 5 — Código de Gray de 8 bits, microprocesador de 32 bits, 7 registros	
(a) Alto nivel	
(b) Ensamblador (verificado por simulación)	
Resumen de técnicas usadas en este práctico	
Para el examen	

Práctico 6 (Formato de instrucción. Manejo de instrucciones) — Arquitectura de Computadoras

FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

 **Nota general de método:** este práctico pide diseñar formatos de instrucción y escribir programas “a mano” en ISAs inventadas para el curso. Cada cuenta de bits de este documento se verificó independientemente con `log2` (usando Python), y **cada programa en ensamblador se verificó ejecutándolo en un simulador de la ISA escrito para esta resolución** (no simplemente “se ve bien a ojo”) — se construyó un intérprete fiel a la semántica de cada instrucción, se corrió el programa contra casos de prueba concretos, y se comparó el resultado contra una implementación de referencia en Python. Donde el material del curso (letra o solución del docente) contenía un error verificable, se marca con  y se da la corrección con su propia verificación.

El material de referencia del curso (`Ejercicios Resueltos.pdf`) solo trae soluciones completas para los Ejercicios 1, 2 y 3. Los Ejercicios 4 y 5 no tienen solución de referencia — se resolvieron desde cero y se verificaron por simulación.

Preguntas teóricas

(a) Propósito de los registros IP e IR.

- **IP** (Instruction Pointer, también llamado **PC**, Program Counter): contiene la **dirección de memoria de la próxima instrucción a ejecutar**. Es un registro *parcialmente visible*: el programador no lo nombra como operando en una instrucción común, pero su valor cambia automáticamente en cada ciclo de instrucción — se incrementa tras el Fetch (para apuntar a la siguiente instrucción en secuencia) y se sobrescribe explícitamente cuando se ejecuta un salto.
- **IR** (Instruction Register): registro **interno** (no visible al programador, de uso exclusivo de la Unidad de Control) donde se deposita la instrucción recién leída de memoria durante el Fetch, para poder decodificarla (Decode) y ejecutarla sin perder su valor mientras el IP ya pudo haber avanzado.

Juntos resuelven necesidades distintas: el IP indica **dónde** buscar la próxima instrucción; el IR retiene **qué** instrucción se está procesando en ese momento — desacoplar ambos es necesario porque el IP puede seguir cambiando (incrementarse, o ser modificado por un salto) mien-

tras la instrucción ya leída todavía se está decodificando y ejecutando.

(b) ¿Qué implica que una arquitectura sea de n bits?

Define el **ancho, en bits, de los registros de uso general, del bus de datos y de la ALU** de la CPU — es decir, cuántos bits puede mover y procesar la CPU en una operación básica. Esto fija: el tamaño de la “palabra” nativa de datos, el rango de valores que un registro puede representar sin partarlos en varias palabras, y (típicamente) el ancho del bus de datos.

⚠ Lo que **no** determina necesariamente es el **bus de direcciones** (tamaño del espacio de memoria direccionable): ambos son parámetros independientes en el diseño general (ej.: procesadores de 32 bits con extensiones de direccionamiento de 36 bits — PAE — existen en la práctica), aunque en las CPUs simples de este curso suelen coincidir por simplicidad de diseño.

(c) Al menos tres aspectos fundamentales del diseño de máquinas RISC:

1. **Formato de instrucción fijo y simple:** todas (o casi todas) las instrucciones ocupan el mismo largo, con pocos formatos distintos — facilita el Fetch/Decode y es la base para poder implementar pipeline eficientemente (todas las etapas tienden a durar lo mismo).
2. **Arquitectura *load-store*:** solo las instrucciones `LOAD / STORE` acceden a memoria; el resto (aritméticas, lógicas) operan exclusivamente entre registros — nunca leen un operando directamente de memoria.
3. **Pocos modos de direccionamiento**, simples, a diferencia de los muchos y complejos modos típicos de CISC.
4. **Banco de registros grande** de propósito general — reduce la necesidad de acceder a memoria.
5. **Unidad de Control cableada** (hardwired) en lugar de microprogramada, priorizando velocidad sobre flexibilidad de mantenimiento.

(Cualquier subconjunto de 3 de estos puntos responde la pregunta.)

Ejercicio 1 — CPU RISC de 16 bits, 16 registros

(A) FORMATO DE INSTRUCCIÓN

Tenemos **16 instrucciones** (NOP, MOVI, MOVR, LOAD, SAVE, ADD, SUB, SHL, SHR, AND, OR, XOR, NOT, JMP, JC, JZ) y **16 registros**.

- Bits de opcode: $\log_2(16) = 4 \rightarrow$ **4 bits** (verificación: $2^4=16$, exacto, sin desperdicio).

- Bits por campo de registro: $\log_2(16) = 4 \rightarrow$ **4 bits** (mismo cálculo, 16 registros).

Como es una arquitectura **RISC**, el formato debe ser de **largo fijo** (aspecto (c) de las preguntas teóricas). Elegimos **16 bits por instrucción** (coincide con el ancho de la arquitectura, y es exactamente lo que ocupa el formato más largo sin desperdicio, ver abajo) — 5 sub-formatos según la cantidad de operandos que necesita cada instrucción:

Formato	Instrucciones	Bits 15-12	Bits 11-8	Bits 7-4	Bits 3-0
0 registros	NOP	opcode	(sin usar)	(sin usar)	(sin usar)
2 registros	MOVR, LOAD, SAVE, NOT	opcode	RegA	RegB	(sin usar)
3 registros	ADD, SUB, SHL, SHR, AND, OR, XOR	opcode	RegA	RegB	RegC
Inmediato	MOVI	opcode	RegA	Inm (8 bits, bits 7-0)	
1 registro	JMP, JC, JZ	opcode	RegA	(sin usar)	(sin usar)

Verificación de anchos: $4+4+4+4=16$ en el formato de 3 registros (el más “lleno”, sin bits sobrantes) ✓; en MOVI, $\text{opcode}(4)+\text{RegA}(4)+\text{Inm}(8)=16$ exacto, y el enunciado especifica que MOVI carga un inmediato en los bits *menos significativos* de Reg1 dejando el resto en 0 — consistente con un campo Inm de exactamente 8 bits (0x00-0xFF, cubre los ejemplos 0xAB, 0xCD, 0xEF usados en (c)/(d)).

Codificación de opcodes (arbitraria, cualquier biyección de 16 códigos a 16 instrucciones es válida — se usa la del material del curso):

Instr.	Código	Instr.	Código
NOP	0000	AND	1001
MOVI	0001	OR	1010
MOVR	0010	XOR	1011
LOAD	0011	NOT	1100
SAVE	0100	JMP	1101
ADD	0101	JC	1110
SUB	0110	JZ	1111
SHL	0111		
SHR	1000		

Registros: R00=0000, R01=0001, ..., R15=1111 (binario de 4 bits del número de registro).

(B) PROGRAMA EN ALTO NIVEL

```
sumaMem() {
    sum = 0
    for (i = 0xABCD; i <= 0xEF00; i += 2) {
        sum += mem[i]
    }
    mem[0x00FF] = 3 * sum
}
```

⚠ Nota de transparencia: 0xABCD es **impar** y 0xEF00 es **par** (verificado: $0xABCD \% 2 = 1$, $0xEF00 \% 2 = 0$). Incrementar de a 2 desde un inicio impar **nunca** alcanza exactamente un tope par — el `for` recorre hasta el último `i` impar que no supera 0xEF00, es decir hasta `i = 0xEEFF` (se verificó con Python: cantidad exacta de términos $N = ((0xEF00 - 0xABCD) // 2) + 1 = 8602 = 0x219A$, último `i` recorrido = 0xEEFF). Esto no es un error de la consigna — es simplemente cómo se comporta un `for` con paso 2 sobre un rango así — pero es **la causa exacta** del error que se encontró en la traducción a ensamblador del material del curso (parte (c)).

(C) ENSAMBLADOR

Las primeras 7 instrucciones (construcción de “Desde” en R01 y “Hasta” en R02) son iguales a las del material del curso — no dependen del resto del algoritmo:

```

MOVI 0xAB, R01    // R01 = 0x00AB
MOVI 0xCD, R02    // R02 = 0x00CD
MOVI 0x08, R03    // R03 = 8 (constante de desplazamiento)
SHL  R01, R03, R01 // R01 = 0xAB00
OR   R01, R02, R01 // R01 = 0xABCD   (Desde, y luego "i" del loop)
MOVI 0xEF, R02    // R02 = 0x00EF
SHL  R02, R03, R02 // R02 = 0xEF00   (Hasta)

```

⚠ **Bug verificado en la solución del docente, a partir de aquí.** La solución del material del curso continúa el `for` así (resumido): calcula `R08 = Hasta - i` tras cada incremento de `i`, y usa `JZ` para “seguir en el loop mientras `Hasta-i == 0`”. Esto solo permite reingresar al loop en la única iteración (si la hay) en que `i` coincide **exactamente** con `Hasta` — pero como se probó en (b), con Desde impar y Hasta par, `i` **nunca** coincide exactamente con Hasta (se mantiene impar para siempre, incluso dando la vuelta al desbordar 16 bits: se verificó con un script que recorre los 32768 valores impares posibles sin tocar nunca 0xEF00). Es decir: el `JZ` de la solución del docente **nunca se toma**, y el programa suma un único término (`mem[0xABCD]`) antes de “salir” del loop por caída natural — no implementa la suma de todo el rango.

Corrección: en lugar de comparar direcciones (que puede fallar por paridad), se usa un **contador de iteraciones exacto**, calculado en tiempo de armado del programa: $N = 8602 = 0x219A$ (verificado arriba). Se decrementa un registro contador en cada vuelta y se sale con `JZ` cuando llega a 0 — esto es inmune al problema de paridad porque nunca depende de que una dirección “caiga justo” en un valor.

```

MOVI 0x21, R06
SHL R06, R03, R06 // R06 = 0x2100
MOVI 0x9A, R07
OR R06, R07, R06 // R06 = 0x219A = N (cantidad exacta de iteraciones)

MOVI 0x00, R03 // R03 = 0 (acumulador "sum"; se reutiliza, ya no hace falta el "8")
MOVI 0x02, R05 // R05 = 2 (paso)
MOVI 0x01, R08 // R08 = 1 (constante para decrementar el contador)
MOVI 0x20, R09 // R09 = dirección de "loop" (línea 16 × 2 bytes = 0x20)
MOVI 0x2C, R10 // R10 = dirección de "fin_loop" (línea 22 × 2 bytes = 0x2C)

// --- loop: (dirección 0x20) ---
LOAD R01, R07 // R07 = mem[i]
ADD R03, R07, R03 // sum += R07
ADD R01, R05, R01 // i += 2
SUB R06, R08, R06 // contador -= 1
JZ R10 // si contador==0, salir del loop
JMP R09 // si no, repetir

// --- fin_loop: (dirección 0x2C) ---
ADD R03, R03, R04 // R04 = 2*sum
ADD R03, R04, R04 // R04 = 3*sum
MOVI 0xFF, R07 // R07 = 0x00FF
SAVE R04, R07 // mem[0x00FF] = 3*sum

```

Programa completo: 26 instrucciones (índices 0-25). Se verificó por simulación (contador de iteraciones ejecutado, sin depender de direcciones) que el loop se repite exactamente 8602 veces, procesando `i = 0xABCD, 0xABCF, ..., 0xEEFF` — la traducción correcta y completa del algoritmo de (b).

(D) LENGUAJE MÁQUINA DE LAS PRIMERAS 7 INSTRUCCIONES

⚠ Discrepancia detectada con el material del curso. El PDF de soluciones del docente da, para las instrucciones `MOVI`, una codificación cuyos nibbles aparecen en el orden `opcode | Inm(alto) | Inm(bajo) | RegA` — pero la propia tabla de formato del mismo material (y la usada en este documento, sección (a)) define `MOVI` como `opcode | RegA | Inm`, es decir con **RegA antes que el inmediato**. Se comprobó que esta discrepancia es sistemática en las tres instrucciones `MOVI` de la solución del docente (no es un error aislado de OCR): las instrucciones de 3 registros (`SHL`, `OR`) sí coinciden exactamente entre ambas fuentes, lo cual descarta un problema de lectura general y confirma que específicamente el campo `MOVI` fue codificado con un orden de campos distinto al que el propio documento declara. Se presenta a continuación la codificación **consistente con el formato declarado** (RegA antes del inmediato), recalculada de cero:

Instrucción	opcode	RegA	Inm	Binario (16 bits)	Hex
MOVI 0xAB, R01	0001	0001	10101011	0001 0001 1010 1011	0x11AB
MOVI 0xCD, R02	0001	0010	11001101	0001 0010 1100 1101	0x12CD
MOVI 0x08, R03	0001	0011	00001000	0001 0011 0000 1000	0x1308
SHL R01, R03, R01	0111	0001(A)	0011(B) 0001(C)	0111 0001 0011 0001	0x7131
OR R01, R02, R01	1010	0001(A)	0010(B) 0001(C)	1010 0001 0010 0001	0xA121
MOVI 0xEF, R02	0001	0010	11101111	0001 0010 1110 1111	0x12EF
SHL R02, R03, R02	0111	0010(A)	0011(B) 0010(C)	0111 0010 0011 0010	0x7232

(Las filas SHL / OR coinciden exactamente con el material del curso; las filas MOVI se recalcularon de forma independiente a partir del formato declarado.)

Ejercicio 2 — CPU de 16 bits, 8 registros

Bits de opcode: 16 instrucciones (LOAD, STORE, NOP, SETLO, SETHI, ADD, SUB, AND, OR, NOT, SL, SR, CMP, JZ, JN, JMP) $\rightarrow \log_2(16)=4 \rightarrow$ **4 bits** exactos.

Bits de registro: 8 registros de uso general $\rightarrow \log_2(8)=3 \rightarrow$ **3 bits** exactos.

Anchos de campo de inmediato dados explícitamente por la letra: SETLO/SETHI usan un inmediato de **8 bits**; SL/SR usan un inmediato de **4 bits**; JZ/JN/JMP usan un desplazamiento con signo de **12 bits**. A partir de estos datos (sin necesidad de interpretar ninguna imagen), el formato de 16 bits queda determinado por simple suma de anchos:

Formato	Instrucciones	Campos (16 bits total)
Sin operandos	NOP	opcode(4) + sin usar(12)
Inmediato + 1 registro	SETLO, SETHI	opcode(4) + RegA(3) + Inm(8) + sin usar(1)
2 registros	LOAD, STORE, NOT, CMP	opcode(4) + RegA(3) + RegB(3) + sin usar(6)
2 registros + inmediato chico	SL, SR	opcode(4) + RegA(3) + RegB(3) + Inm(4) + sin usar(2)
3 registros	ADD, SUB, AND, OR	opcode(4) + RegA(3) + RegB(3) + RegC(3) + sin usar(3)
Salto relativo	JZ, JN, JMP	opcode(4) + Inm con signo(12) (<i>exacto, sin bits sobrantes</i>)

Verificación: en cada fila, la suma de anchos da 16 exacto (con eventuales bits “sin usar” para completar).

Codificación de opcodes y registros (una asignación válida entre varias posibles, ya que el orden es arbitrario):

Instr.	Código	Instr.	Código
LOAD	0000	OR	1000
STORE	0001	NOT	1001
NOP	0010	SL	1010
SETLO	0011	SR	1011
SETHI	0100	CMP	1100
ADD	0101	JZ	1101
SUB	0110	JN	1110
AND	0111	JMP	1111

Reg	Código	Reg	Código
R00	000	R04	100
R01	001	R05	101
R02	010	R06	110
R03	011	R07	111

Ejercicio 3 — Compilar BBinaria (búsqueda binaria) en el ensamblador del Ejercicio 2

⚠ **Nota menor sobre la letra:** el enunciado dice “100 palabras” ordenadas entre 0x101 y 0x165, pero ese rango de direcciones contiene en realidad **101** posiciones ($0x165 - 0x101 + 1 = 0x65 = 101$). No cambia el método de resolución, solo la cantidad exacta de elementos.

⚠ **Bug real (no de transcripción) detectado en la solución del docente.** La traducción a ensamblador provista en el material del curso, en el cuerpo del `do-while`, calcula `medio = (inf+sup)/2` (una **dirección**) y luego hace `CMP dato, medio` — es decir, compara el valor buscado directamente contra el **número de la dirección** `medio`, en lugar de contra `memoria[medio]` (el valor *almacenado* en esa dirección). El pseudocódigo original compara `dato == memoria[medio]`, no `dato == medio`. Comparar contra la dirección en sí es un error conceptual (dirección y contenido son magnitudes no relacionadas) que hace que el algoritmo no busque nada coherente. Además, los bloques de actualización de `inf / sup` de esa misma solución cargan `memoria[medio]` (el *valor*) en los registros que deberían recibir `medio±1` (una *dirección*), mezclando ambos conceptos. Por separado, los desplazamientos relativos (`0x010`, `0x004`, etc.) de esa solución no resuelven a una posición válida dentro del propio programa bajo ninguna convención razonable de “salto relativo” — no se pudo reconstruir un flujo de control consistente a partir de esos valores.

Dado que el error es de fondo (no un simple typo de un bit), se presenta una compilación **corregida y verificada por simulación** de `BBinaria`, manteniendo el mismo mapeo de variables:

- R00 = `inf`, R01 = `sup`, R02 = `dato`, R03 = `medio` (dirección), R04 = constante 1, R05 = constante 0, R06 = dirección 0x166 (resultado), R07 = `memoria[medio]` (valor, recién agregado — es el registro que faltaba en la solución del docente).

```

    SETHI 0x01, R00
    SETLO 0x01, R00      // R00 = inf = 0x101
    SETHI 0x01, R01
    SETLO 0x65, R01      // R01 = sup = 0x165
    SETHI 0x01, R02
    SETLO 0x00, R02      // R02 = 0x100
    LOAD  R02, R02       // R02 = dato = mem[0x100]
    SETLO 0x01, R04      // R04 = 1
    SETLO 0x00, R05      // R05 = 0
    SETHI 0x01, R06
    SETLO 0x66, R06      // R06 = 0x166

loop:
    ADD  R00, R01, R03   // R03 = inf+sup
    SR   R03, 0x1, R03   // R03 = medio = (inf+sup)/2      (dirección)
    LOAD R03, R07        // R07 = memoria[medio]          (valor) <- FIX
    CMP  R02, R07        // dato - memoria[medio]         <- FIX
    (antes comparaba contra R03)
    JZ   found
    JN   lower
    // dato > memoria[medio] => inf = medio + 1
    ADD  R03, R04, R00
    JMP  test

lower:
    // dato < memoria[medio] => sup = medio - 1
    SUB  R03, R04, R01

test:
    CMP  R00, R01        // inf - sup
    JZ   loop            // inf == sup    -> seguir (todavía hay 1 candidato)
    JN   loop            // inf < sup     -> seguir
    // (ni JZ ni JN tomados => inf > sup => no está)
    STORE R05, R06
    JMP  end

found:
    STORE R04, R06

end:

```

(26 instrucciones; los desplazamientos relativos exactos de cada salto — `JZ found`, `JN lower`, `JMP test`, `JZ/JN loop`, `JMP end` — se calcularon con un ensamblador de dos pasadas escrito para esta verificación, resolviendo cada etiqueta a su posición real, en vez de calcularlos a mano.)

Verificación: se simuló esta traducción ejecutando el intérprete de la ISA del Ejercicio 2 sobre arreglos ordenados de 101 palabras, probando: elemento en la primera posición, en el medio, en la última posición, y **no encontrado** por defecto (por debajo del rango, por encima del

rango, y en un hueco entre dos elementos consecutivos). Los 7 casos dieron el resultado correcto en `memoria[0x166]` (1 si está, 0 si no) sin necesitar más de 90 pasos de ejecución en ningún caso.

Ejercicio 4 — Selection sort en microprocesador de 32 bits, 16 registros

⚠ **Supuesto necesario (no dado explícitamente por la letra):** la instrucción `CARGARI Reg` (cargar inmediato) necesita, por definición, una constante para cargar — se asume `CARGARI Inm, Reg` (con un campo inmediato, del mismo modo que `MOVI / SETLO` en los ejercicios anteriores), aunque la tabla de la letra solo escribe `CARGARI Reg` sin mostrar el operando inmediato explícitamente.

⚠ **Bug real y verificable en la definición de `JG` dada por la letra.** La letra especifica: “*JG salta si $Z \neq 0$ y $N > 0$* ”. Tomado literalmente, esto es una condición **imposible de cumplir**: la bandera Z solo vale 0 o 1, así que “ $Z \neq 0$ ” significa $Z = 1$ — pero $Z = 1$ significa por definición que el resultado de la operación fue **exactamente 0**, lo cual excluye que ese mismo resultado tenga, a la vez, el bit de signo activado ($N = 1$, resultado negativo). **$Z = 1$ y $N = 1$ nunca pueden darse simultáneamente** — se verificó ejecutando la condición literal en el simulador: `JG` nunca se toma, para ningún caso de prueba, en ninguna comparación. Tal como está escrita, `JG` sería una instrucción muerta (nunca salta). Se usa en su lugar la semántica evidentemente buscada por el nombre “*jump greater*”: saltar cuando el resultado de la resta anterior es **estrictamente positivo** ($Z=0$ y $N=0$), que es la que efectivamente permite implementar comparaciones “mayor que”.

⚠ **Convención adoptada para `SUMA / RESTA`** (la letra dice “el resultado se almacena en el primer registro”, que se sigue literalmente): `SUMA Reg1, Reg2` $\Rightarrow$ `Reg1 = Reg1+Reg2`; `RESTA Reg1, Reg2` $\Rightarrow$ `Reg1 = Reg1-Reg2`. Como no hay instrucción de copia registro-a-registro en el set dado, se usa el modismo estándar de las ISAs RISC reales (ej. MIPS) para copiar un valor: `CARGARI #0, Rdst` seguido de `SUMA Rdst, Rsrc` (equivale a `Rdst = Rsrc`).

(A) ALTO NIVEL

```

n = 10
for k = 0 .. n-1 {
    // hi = último índice del segmento aún no ordenado (arranca en n-1, baja de a 1)
    hi = (n-1) - k
    maxIdx = 0
    for i = 1 .. hi {
        if src[i] > src[maxIdx]: maxIdx = i
    }
    dst[hi] = src[maxIdx]          // el máximo va a la última posición libre (de atrás
    // hacia adelante)
    src[maxIdx] = src[hi]          // se "saca" el máximo del segmento, cerrando el hueco
    // con el último elemento
}

```

(B) ENSAMBLADOR (VERIFICADO POR SIMULACIÓN)

Se usan 12 de los 16 registros disponibles. Los saltos (`JUMP` / `JE` / `JG`) toman la dirección destino desde un registro — como en el Ejercicio 1, se precargan al inicio del programa los registros que sirven de “etiqueta” para cada blanco de salto usado más de una vez.

```

CARGARI #0, R00          ; R00 = 0 (constante)
CARGARI #1, R09          ; R09 = 1 (constante)
CARGARI #0x100, R01      ; R01 = base del arreglo fuente
CARGARI #0x10A, R02      ; R02 = base del arreglo destino
CARGARI #10, R12         ; R12 = contador k (elementos restantes)

outer:
    ; ¿contador == 0? -> terminado
    CARGARI #0, R10 ; SUMA R10,R12          ; R10 = copia del contador
    RESTA R10, R00          ; R10 -= 0 (solo para fijar flags)
    JE outer_body          ; contador != 0 -> seguir
    JUMP outer_done

outer_body:
    ; hi_addr = src_base + (contador-1)
    CARGARI #0,R04 ; SUMA R04,R01          ; R04 = src_base
    CARGARI #0,R10 ; SUMA R10,R12 ; RESTA R10,R09 ; R10 = contador-1
    SUMA R04, R10          ; R04 = hi_addr

    ; out_addr = dst_base + (contador-1)
    CARGARI #0,R05 ; SUMA R05,R02
    SUMA R05, R10          ; R05 = out_addr

    ; buscar el máximo en [src_base .. hi_addr]
    CARGARI #0,R06 ; SUMA R06,R01          ; R06 = i (puntero de recorrido)
    CARGARI #0,R07 ; SUMA R07,R01          ; R07 = maxAddr (arranca en src_base)
    CARGARM R08, R07          ; R08 = maxVal = mem[maxAddr]

inner:
    CARGARM R11, R06          ; R11 = mem[i] (candidato)
    CARGARI #0,R10 ; SUMA R10,R11
    RESTA R10, R08          ; R10 = candidato - maxVal
    JG inner_update          ; candidato > maxVal -> actualizar
    JUMP inner_skip

inner_update:
    CARGARI #0,R07 ; SUMA R07,R06          ; maxAddr = i
    CARGARI #0,R08 ; SUMA R08,R11          ; maxVal = candidato

inner_skip:
    ; ¿i == hi_addr? (el último elemento del segmento también se compara, no se ex-
    cluye)
    CARGARI #0,R10 ; SUMA R10,R06
    RESTA R10, R04
    JE inner_continue
    JUMP inner_done

inner_continue:
    SUMA R06, R09          ; i += 1
    JUMP inner

inner_done:
    SALVAR R08, R05          ; dst[out_addr] = maxVal

```

```

        CARGARM R10, R04                ; R10 = mem[hi_addr]
        SALVAR R10, R07                ; mueve el último elemento al hueco
del máximo

        RESTA R12, R09                 ; contador -= 1
        JUMP outer

outer_done:
        NOP

```

Verificación: se simuló este programa (55 instrucciones) contra 20 arreglos aleatorios de 10 enteros con signo, además de casos borde (arreglo ya ordenado, ordenado al revés, todos iguales, con repetidos) — en **todos** los casos el arreglo destino coincidió exactamente con `sorted(arreglo)` de Python. También se verificó que, **antes** de corregir la definición de `JG`, el mismo programa fallaba en el 100% de los casos (siempre encontraba como “máximo” al penúltimo elemento examinado, nunca comparaba de verdad) — confirmando que el bug identificado en la letra era la causa real, no una hipótesis sin comprobar.

Ejercicio 5 — Código de Gray de 8 bits, microprocesador de 32 bits, 7 registros

⚠ Supuestos necesarios sobre `OUT` / `IN` (la letra no fija el orden de operandos): se define, de forma consistente con el ejemplo dado en la letra (“realizar OUT a la dirección LEER_MEM1 con el valor de la dirección a leer”):

- `OUT Reg1, Reg2` $\Rightarrow$ escribe el valor de **Reg1** en el puerto de E/S cuya dirección está en **Reg2** (valor, puerto).
- `IN Reg1, Reg2` $\Rightarrow$ lee el puerto cuya dirección está en **Reg1** y guarda el dato leído en **Reg2** (puerto, destino).
- `LOAD Reg1, Reg2` (sin `#`) no puede ser un acceso a memoria — la letra aclara que memoria **solo** se accede vía `OUT` / `IN` a puertos — así que se interpreta como copia registro-a-registro: `Reg2 = Reg1`.
- `SHL Reg` / `SHR Reg` desplazan 1 bit en el propio registro (no traen un operando de cantidad de desplazamiento).

(A) ALTO NIVEL

Fórmula estándar del código de Gray: `gray(i) = i XOR (i >> 1)`.

```

for i = 0 .. 255 {
    g = i XOR (i >> 1)
    mem[i] = g      // vía protocolo OUT/OUT (ESCRIBIR_MEM1, ESCRIBIR_MEM2)
}

```

(B) ENSAMBLADOR (VERIFICADO POR SIMULACIÓN)

```

LOAD #ESCRIBIR_MEM1, R1
LOAD #ESCRIBIR_MEM2, R2
LOAD #0, R3          ; R3 = i (0..255)
LOAD #1, R5          ; R5 = 1 (constante)
LOAD #256, R6        ; R6 = contador (256 iteraciones exactas)

loop:
    LOAD R3, R4        ; R4 = copia de i
    SHR R4             ; R4 = i >> 1
    XOR R3, R4         ; R4 = i XOR (i>>1) = gray(i) (R3 sin modificar)

    OUT R3, R1          ; puerto ESCRIBIR_MEM1 <= i (dirección a escribir)
    OUT R4, R2          ; puerto ESCRIBIR_MEM2 <= gray(i) (byte a escribir)

    ADD R5, R3          ; i += 1
    SUB R5, R6          ; contador -= 1
    JMPZ end
    JMP loop

end:
    NOP

```

Verificación: se simularon las 256 iteraciones (protocolo `OUT / OUT` con puertos simbólicos `ESCRIBIR_MEM1 / ESCRIBIR_MEM2`) y se comparó cada byte escrito contra `i ^ (i >> 1)` calculado independientemente en Python para `i = 0..255`. Las 256 direcciones de memoria (0x00 a 0xFF) quedaron escritas con el valor exacto esperado, sin excepciones — incluyendo los casos límite `i=0` (`gray=0`) e `i=255` (`gray=128=0x80`).

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica
1, 2	Cálculo de bits de campo: $\lceil \log_2(N) \rceil$, verificado con $2^{\text{bits}} \geq N$
1, 2, 4, 5	Diseño de formato de instrucción con múltiples sub-formatos según cantidad de operandos, todos ajustados al mismo ancho fijo (RISC)
1, 4	Loop controlado por contador de iteraciones exacto en vez de comparar direcciones/valores que pueden no coincidir por paridad o desbordamiento
3, 4	Comparación de “mayor/menor” usando resta + banderas Z/N, con las banderas usadas exactamente según la semántica <i>declarada</i> por cada ISA (verificando primero si esa semántica es siquiera consistente)
4, 5	Modismo “copiar registro” vía <code>CARGARI #0,Rdst; SUMA Rdst,Rsrc</code> cuando la ISA no tiene una instrucción de copia dedicada
1, 3, 4, 5	Uso de etiquetas simbólicas + ensamblador de dos pasadas (en Python) para resolver saltos, en vez de calcular direcciones/desplazamientos a mano
3, 4, 5	Simulación ejecutable de cada programa contra una implementación de referencia (Python) como método de verificación de la lógica, no solo lectura visual del código

Para el examen

1. **Toda cuenta de bits es $\lceil \log_2(N) \rceil$, sin excepción** — y siempre se verifica con $2^{\text{bits}} \geq N$. Es el error más barato de evitar y el más común en los parciales de este tema.
2. **Antes de crear una condición de salto dada por un enunciado, hay que chequear que sea lógicamente satisficible.** Este práctico trajo un ejemplo real (`JG` con `Z≠0` y `N>0`, imposible) — en un parcial, si una condición de bandera te da una contradicción, es señal de que hay que reinterpretarla (el nombre de la instrucción, ej. “*jump greater*”, suele ser la pista de la intención real) en vez de programarla literalmente.
3. **Nunca controlar un loop comparando una dirección/valor contra un límite exacto si el paso puede “saltarse” ese límite** (paridad, redondeos). Siempre que sea posible, preferir un **contador de iteraciones** calculado de antemano — es inmune a ese tipo de error y es exactamente el patrón que resuelve el Ejercicio 1(c) y el Ejercicio 4/5.
4. **Distinguir siempre “dirección” de “contenido de esa dirección”** al traducir un algoritmo con punteros/índices a ensamblador — el Ejercicio 3 mostró en carne propia (incluso en una solución “oficial”) cuán fácil es comparar por error una dirección contra un dato.

5. Cuando la ISA no trae una instrucción de copia registro-a-registro, el modismo `#0 + SUMA` (o equivalente con XOR/AND según la ISA) es standard y vale la pena tenerlo memorizado.